Aether — Configuration Drift Detection

Know When Reality Diverges from Intent

Detect spec-vs-live-state divergence across runtime, image, resources, network, scaling, and health — then auto-reconcile with a single flag.

Detect • Report • Reconcile • Audit

What is Drift?

Configuration drift occurs when the live state of a workload diverges from the desired state defined in its YAML spec. Drift is silent, cumulative, and dangerous.

YAML Spec (Desired) ≠ Live State (Actual) → DRIFT DETECTED

How Drift Happens

Manual Changes

An operator scales replicas via `kubectl` directly, bypassing the Aether spec. The spec says 3 replicas, but the cluster runs 5. This is the most common source of drift.

Failed Updates

A deploy partially succeeds: the image updates but resource limits revert. The spec shows the new configuration, but the runtime retains a mix of old and new settings.

Spec Evolution

The team updates the YAML spec but forgets to re-deploy. The spec now requests TLS on ingress, but the running service still serves plain HTTP.

Runtime Auto-Scaling

HPA scales the workload beyond the spec's defined range. Scaling min and max in the spec no longer match the actual replica count managed by the autoscaler.

> *Drift is not a bug — it is the natural entropy of any system with multiple actors and multiple control surfaces. The solution is continuous*

detection and automated reconciliation.

Drift Detection

Run drift detection against any deployed workload. Aether compares the YAML spec, stored state, and live runtime state to produce a comprehensive drift report.

```
$ aether drift my-app

Drift Report:  my-app
Status:        DRIFT DETECTED (CRITICAL)

Drifts (3):
  [WARNING]  runtime (Runtime)
    Expected: kubernetes
    Actual:   podman

  [CRITICAL] ingress.tls (Network)
    Expected: TLS enabled
    Actual:   TLS disabled

  [WARNING]  health (Health)
    Expected: health probes configured
    Actual:   no health probes

Reconciliation Plan (3 actions):
  1. [Redeploy] Migrate from podman to kubernetes (requires restart)
  2. [Redeploy] Reconcile ingress.tls: expected TLS enabled (requires resta
  3. [Redeploy] Reconcile health: expected health probes configured (requir
```

Live Diff View

The live diff report presents a three-column comparison: **Spec** (what the YAML says), **Stored** (what Aether recorded at last deploy), and **Live** (what the runtime reports now). Rows marked with ! indicate differences. This makes it trivial to identify exactly where drift occurred and which layer is responsible.

```
$ aether drift my-app --live-diff

Live Diff: my-app

  Field            Spec                 Stored               Live
  ---------------------------------------------------------------------
! runtime          kubernetes           podman               podman
  image            ghcr.io/app:v2       ghcr.io/app:v2       ghcr.io/app:v2
! replicas         3                    1                    1
  ready            -                    -                    true

  2 difference(s) found.
```

Auto-Reconciliation

Aether does not just detect drift — it fixes it. The `--reconcile` flag executes the reconciliation plan automatically, bringing live state back in sync with the spec.

```
$ aether drift my-app --reconcile

Drift Report:  my-app
Status:        DRIFT DETECTED (WARNING)

Executing Reconciliation Plan:
  1. [Redeploy]     Migrate from podman to kubernetes ... OK
  2. [Update Image] Update image to ghcr.io/app:v2.1  ... OK
  3. [Scale]        Fix scaling: minReplicas < maxReplicas ... OK (advisory

All 3 actions completed successfully.
Workload 'my-app' is now in sync with spec.
```

Reconciliation Actions

Action Type	Behavior	Restart Required	Risk Level
Redeploy	Stop old instance, rebuild from spec, run new instance	Yes	Warning
Update Image	Pull new image, replace running container	Yes	Info
Update Config	Patch configuration in-place without restarting	No	Info
Scale	Adjust replica count (logged as advisory for HPA)	No	Info
None	No runtime action needed, informational only	No	Info

Reconciliation follows the same build-and-deploy pipeline as a fresh deploy: stop the old instance, rebuild from the spec, run the new instance, and update the state store. This guarantees consistency.

Drift Categories

Aether classifies every drift by category and severity, enabling teams to prioritize fixes and build alerting rules around the types that matter most.

Runtime Drift

Spec says Kubernetes but workload runs on Podman. Severity: WARNING. Reconciliation: full migration via the Aether migrate engine.

Image Drift

Running image tag does not match the expected registry/name:tag. Severity: INFO. Reconciliation: pull and redeploy the correct image.

Resource Drift

CPU or memory allocation exceeds reasonable thresholds (>32 cores, >128 Gi). Severity: WARNING. Flags over-provisioned workloads for right-sizing.

Network Drift

Service defined without ports, or ingress enabled without TLS. Severity: CRITICAL for missing TLS. Flags security-critical misconfigurations.

Scaling Drift

Autoscaling enabled but minReplicas >= maxReplicas, making the autoscaler non-functional. Severity: WARNING. Reconciliation: fix scaling bounds.

Health Drift

Service-enabled workload has no health probes configured. Severity: WARNING. Without probes, failed instances are not detected or restarted.

Integration with Audit Trail

Every drift detection scan and every reconciliation action generates an audit event with SHA-256 integrity hash. The audit trail records who ran the drift check, what drifts were found, and which reconciliation actions were executed. This provides a complete compliance record showing that drift was detected, reported, and remediated within the required SLA window.

```
# View drift-related audit events
$ aether audit --filter drift,reconcile

2026-04-12T10:30:00Z  admin  drift-scan     my-app  3 drifts found
2026-04-12T10:30:05Z  admin  reconcile      my-app  3 actions executed
2026-04-12T10:30:10Z  admin  drift-scan     my-app  0 drifts (in sync)
```

Eliminate Drift

Aether makes configuration drift visible, actionable, and fixable — across every runtime, every workload, every time.

7	3	5
Drift Categories	Severity Levels	Reconciliation Actions

From Detection to Resolution in Seconds

Detect drift, review the reconciliation plan, and auto-fix with `--reconcile`. Every action is logged, audited, and traceable.

```
# Complete drift workflow
$ aether drift my-app                      # detect
$ aether drift my-app --live-diff           # compare
$ aether drift my-app --reconcile           # fix
$ aether audit --filter drift,reconcile     # verify
```